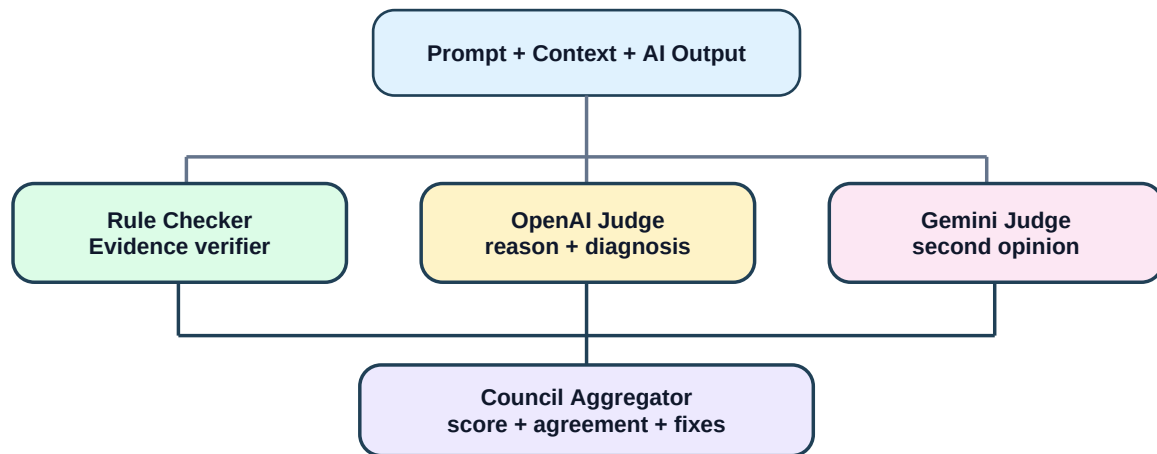


AI Diagnosis Council

A multi-model AI debugging and evaluation platform that tells developers what failed, why it failed, and how to fix it.



Prepared for a 2-month Polaris MVP and pitch plan

1. Core Idea

AI Diagnosis Council is not just another AI evaluation dashboard. It uses multiple judge models and rule-based validators like a council of experts. Each judge has a role: factual support, reasoning quality, prompt quality, citation quality, safety, and fix recommendations.

The product goal is simple: reduce AI debugging time from hours to minutes by turning failed AI outputs into clear diagnosis reports.

Normal evaluator	AI Diagnosis Council
Gives a hallucination or quality score.	Gives score, evidence, likely cause, confidence, and fixes.
Usually one model or one metric.	Multiple model judges plus rule-based checks.
Answers: What failed?	Answers: What failed, why, and what should change?

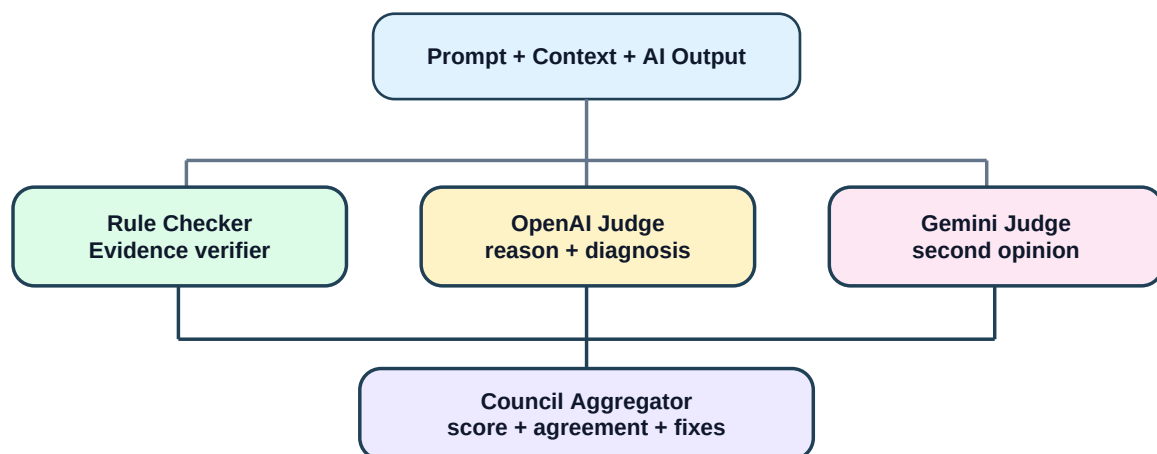
2. MVP Scope for 2 Months

Start with document-based AI systems, also called RAG systems, because they provide a source of truth. The MVP should not try to judge open-ended future questions like World War III predictions. It should judge whether an answer is supported by the provided context.

MVP input: user question, system prompt, retrieved context or uploaded document, AI answer, optional expected answer.

MVP output: hallucination score, reasoning score, citation/support score, instruction-following score, supported claims, unsupported claims, likely root cause, and recommended fixes.

3. Product Architecture



Recommended stack: Next.js for frontend, FastAPI for backend, PostgreSQL or Supabase for database, OpenAI as primary judge, Gemini as second judge, and optional Claude later for long-context analysis. Use a rule engine for exact contradictions and evidence matching.

4. Council Roles

Judge	Responsibility	Example output
Evidence Judge	Checks whether answer claims are supported by context.	Claim unsupported: refund is 30 days.
Reasoning Judge	Checks logic, arithmetic, and chain of reasoning.	Model used wrong calculation.
Prompt Judge	Checks prompt ambiguity or conflicting instructions.	Prompt did not force context-only answer.
Retrieval Judge	Checks whether correct information existed in retrieved context.	Correct document missing, likely retrieval failure.
Fix Judge	Suggests changes to prompt, chunks, citations, or retrieval.	Add citation requirement and reduce chunk size.

5. How Root Cause Analysis Works

The system should not pretend to know the exact cause with 100 percent certainty. It should gather evidence and estimate the most likely cause, similar to how a doctor diagnoses from symptoms and tests.

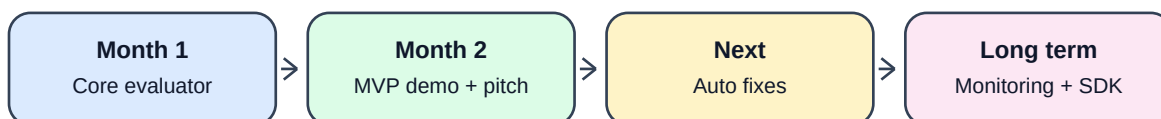
Rule example: If the correct fact exists in context but the answer contradicts it, retrieval probably worked and the failure is likely model generation or instruction following. If the correct fact is missing from context, retrieval failure is likely.

The product should show confidence as evidence-based, not magical. Example: 2 of 3 judges agree, and the rule checker found a contradiction, so confidence is high.

6. Future Evaluation Scores

Evaluation is one module inside the bigger platform. Later, you can add more scores: emotional intelligence, confidence calibration, helpfulness, consistency, completeness, safety, bias, citation quality, and task success.

7. Two-Month Roadmap



Weeks 1-2: build form input, backend API, database schema, and OpenAI judge. **Weeks 3-4:** add claim extraction, evidence checker, Gemini second judge, and scoring. **Weeks 5-6:** dashboard, diagnosis reports, PDF export, test examples. **Weeks 7-8:** polish UI, make demo datasets, build landing page, prepare Polaris pitch.

8. Polaris Pitch Positioning

Pitch line: AI teams do not only need to know that an answer failed. They need to know why it failed and what to change. AI Diagnosis Council uses multiple judge models plus evidence checks to diagnose hallucinations, prompt issues, retrieval failures, and reasoning errors.

The first market is students, indie hackers, and early AI app builders making RAG chatbots, internal knowledge bots, and agent workflows. The wedge is a simple debugging report for prompt + context + output.

9. Claude / Codex MVP Build Prompt

Build a full-stack MVP called AI Diagnosis Council.

Goal:

Create a web app that helps developers debug AI outputs. The user submits a prompt, retrieved context or document text, and an AI answer. The app returns an evaluation and diagnosis report.

Tech stack:

- Frontend: Next.js with TypeScript
- Backend: FastAPI with Python
- Database: PostgreSQL or Supabase
- APIs: OpenAI as primary judge, Gemini as second judge
- Optional later: Claude judge

Core features:

1. User can create a new evaluation case.
2. Input fields: user question, system prompt, context, AI answer, optional expected answer.
3. Backend extracts key claims from the AI answer.
4. Rule checker compares claims against context and marks supported, unsupported, or contradicted.
5. OpenAI judge returns hallucination score, reasoning score, likely cause, and fixes in strict JSON.
6. Gemini judge gives second opinion in strict JSON.
7. Aggregator combines rule checker + judges into final scores, agreement level, confidence, root cause, and recommended fixes.
8. Dashboard shows report cards, claim table, root-cause breakdown, and fix suggestions.
9. Export report as markdown or PDF.
10. Include sample demo cases for refund policy, medical policy, college FAQ, and product support.

Important behavior:

- Do not claim exact truth when no context exists.
- If context is missing, say this is not verifiable and evaluate evidence quality instead.
- Show confidence based on judge agreement and rule-check evidence.
- Keep code modular: judges/, rules/, aggregator/, reports/, api/.

Deliverables:

- Working local setup
- README with installation steps
- .env.example for API keys
- Clean UI
- Demo data
- Tests for rule checker and aggregator

10. Future Full Platform Plan

After the MVP, the product can grow into an AI Quality Engineering Platform: evaluation, diagnosis, auto-fix generation, prompt experiments, RAG debugging, model comparison, production monitoring, and SDK-based tracing.

Long-term modules: multi-model judge marketplace, auto-generated test cases, prompt versioning, hallucination monitoring, cost and latency tracking, confidence calibration, and team dashboards.

11. Demo Script for Polaris

Start with a broken AI answer. Show the answer says refund is 30 days while context says refund is 7 days. Then run AI Diagnosis Council. The report should show contradiction, hallucination score, likely cause, model ignored context, judge agreement, and recommended fixes. End by saying: current tools show traces; we turn failures into diagnosis and action.

12. Final Founder Focus

Do not build everything at once. Build one strong wedge: document-based hallucination and root-cause debugging. Make the demo extremely clear. If the demo feels magical to a developer, the pitch becomes strong.